360 Vision FSD Pipeline Explained

nuScenes data access, calibration, LiDAR projection, BEV, and visualization

Project: vision-fsd / fsd package

Generated from docs/360_vision_fsd_explained.md

This document explains every current 360-vision component in fsd/:

data loading, scene sequencing, camera sweeps, coordinate transforms,

LiDAR-to-camera projection, ego-frame BEV rendering, and the next

occupancy-grid step. The dataset remains external at D:/nuscenes.

`Canonical visualization entry point: python -m fsd.visualize`

This document explains the current `fsd/` 360-vision pipeline in the project.

It is written as a teaching document: what each file does, how nuScenes is sequenced, how camera and LiDAR records are connected, how LiDAR points are projected into camera images, how the ego-frame BEV is rendered, and why this is the correct stepping stone toward occupancy mapping.

The code covered here lives in:

```
fsd/
  __init__.py
  data.py
  contact_sheet.py
  lidar_projection.py
  bev.py
  visualize.py
```

The dataset is read directly from:

```
D:/nuscenes
```

No nuScenes images, LiDAR files, sweeps, maps, or metadata are copied into the project folder or into `C:/`.

1. Big Picture

The old monocular pipeline is video-centric:

```
video.mp4
  frame 0
  frame 1
  frame 2
  frame 3
```

Each frame is one image from one camera. The code reads the next video frame, runs perception, draws overlays, and moves on.

The new 360 pipeline is dataset-centric and sensor-centric:

```
nuScenes scene
  sample 0
    six camera images
    LiDAR_TOP point cloud
    radar files
    ego pose
    calibration

  sample 1
    six camera images
    LiDAR_TOP point cloud
    radar files
    ego pose
    calibration
```

In nuScenes, the closest concept to a video is a `scene`. A scene is a short driving segment, around 20 seconds. The closest concept to a video frame is a `sample`. A sample is a synchronized key moment in the scene.

So the mental mapping is:

```
monocular video.mp4      ~= nuScenes scene
monocular video frame    ~= nuScenes sample
camera image             ~= sample_data record
next video frame         ~= sample["next"]
```

The pipeline we are building is:

```
nuScenes scene
  -> synchronized six-camera access
  -> LiDAR/camera calibration validation
  -> ego-frame LiDAR BEV
  -> occupancy grid
  -> temporal occupancy fusion
```

This follows the Phase 1 plan in `plan.md`: world modeling first, with a classical and inspectable baseline before using large neural BEV models.

2. Why We Started With Geometry Before Neural BEV Models

Models like BEVFormer, BEVDepth, BEVFusion, MapTR, SurroundOcc, Occ3D, and SparseOcc are important references. But they all assume that the data pipeline is correct:

```
six camera images
camera intrinsics
camera-to-ego extrinsics
ego poses
timestamps
BEV coordinate convention
sensor synchronization
```

If those are wrong, a neural model can fail in ways that are hard to debug. Bad output could come from bad preprocessing, bad coordinate transforms, bad timestamp alignment, wrong axes, wrong camera order, wrong resizing, or the model itself.

So the project starts with a geometric baseline:

```
load sensors directly
validate calibration visually
project LiDAR into cameras
render ego-frame BEV
then build occupancy
```

This creates a trusted scaffold. Neural models can be integrated later and compared against something we understand.

3. nuScenes Structure

The dataset root looks like this:

```
D:/nuscenes/
  samples/
    CAM_FRONT/
    CAM_FRONT_LEFT/
    CAM_FRONT_RIGHT/
    CAM_BACK/
    CAM_BACK_LEFT/
    CAM_BACK_RIGHT/
    LIDAR_TOP/
    RADAR_*/

  sweeps/
    CAM_FRONT/
    CAM_FRONT_LEFT/
    ...
    LIDAR_TOP/

  maps/

  v1.0-trainval/
    scene.json
    sample.json
    sample_data.json
    calibrated_sensor.json
    sensor.json
    ego_pose.json
    sample_annotation.json
```

The actual sensor files are stored under `samples/` and `sweeps/`. The JSON metadata tells us how those files connect.

Important tables:

`scene.json`

Lists driving segments. A scene has a first sample token, a last sample token, and a number of samples.

`sample.json`

Lists keyframes. Each sample has:

`token`

`timestamp`

`prev`

`next`

`scene_token`

The `next` field is how we walk through a scene.

```
sample_data.json
```

Lists actual sensor records. Each record points to one file, such as a camera JPEG or LiDAR `.pcd.bin`.

Important fields:

```
token
sample_token
ego_pose_token
calibrated_sensor_token
timestamp
filename
is_key_frame
prev
next
```

```
sensor.json
```

Maps sensor tokens to channels such as `CAM_FRONT` or `LIDAR_TOP`.

```
calibrated_sensor.json
```

Stores each sensor's extrinsics relative to the ego vehicle:

```
translation
rotation
camera_intrinsic  # for cameras only
```

```
ego_pose.json
```

Stores the ego vehicle pose in the global/world frame at a sensor timestamp.

4. Samples vs Sweeps

nuScenes has two kinds of temporal data:

```
samples = sparse synchronized keyframes, around 2 Hz
sweeps  = intermediate sensor captures between keyframes
```

Key samples are easy to work with because they are the main synchronized packets:

```
sample 0
  CAM_FRONT key image
  CAM_FRONT_LEFT key image
  ...
  LIDAR_TOP key scan
```

Sweeps provide smoother temporal resolution:

```
sample 0
  sweep 0.1
  sweep 0.2
  sweep 0.3
sample 1
```

But sweeps are not guaranteed to be perfectly synchronized across sensors.

For smooth camera visualization we group the nearest six camera images around the `CAM_FRONT` timestamp. For LiDAR overlays and BEV we currently use key samples, because those are cleaner and easier to validate.

Example from scene 0:

```
key samples: 40
camera sweep groups: 233
```

At 2 FPS, 40 key samples are around 20 seconds. At 12 FPS, 233 camera sweep groups are around 19.4 seconds. That is why the sweep video looks smoother: it uses real intermediate camera frames, not just keyframes played faster.

5. `fsd/data.py`

This is the dataset access layer. It reads nuScenes metadata directly and constructs Python objects for the rest of the pipeline.

5.1 Camera channels

The six 360 cameras are:

```
CAMERA_CHANNELS = (
    "CAM_FRONT_LEFT",
    "CAM_FRONT",
    "CAM_FRONT_RIGHT",
    "CAM_BACK_LEFT",
    "CAM_BACK",
    "CAM_BACK_RIGHT",
)
```

The code uses this order for contact sheets and videos:

```
CAM_FRONT_LEFT | CAM_FRONT | CAM_FRONT_RIGHT
CAM_BACK_LEFT  | CAM_BACK  | CAM_BACK_RIGHT
```

5.2 `CameraFrame`

One `CameraFrame` represents one camera image at one timestamp:

```
CameraFrame(
    channel,
    image_path,
    timestamp_us,
    sample_data_token,
    calibrated_sensor,
    ego_pose,
    is_key_frame,
)
```

It contains:

```
channel             e.g. CAM_FRONT
image_path          path to JPEG on D:/nuscenes
timestamp_us        timestamp in microseconds
sample_data_token   nuScenes record token
calibrated_sensor   camera extrinsics and intrinsics
ego_pose            vehicle pose at this camera timestamp
is_key_frame        whether this is from samples/ or sweeps/
```

Camera intrinsics are available through:

```
camera.camera_intrinsic
```

The intrinsic matrix is:

```
K = [ fx   0  cx
      0   fy  cy
      0    0   1 ]
```

This maps 3D camera-frame points into image pixels.

5.3 `LidarFrame`

One `LidarFrame` represents one `LIDAR_TOP` scan:

```
LidarFrame(
    channel,
    pointcloud_path,
    timestamp_us,
    sample_data_token,
    calibrated_sensor,
    ego_pose,
)
```

It contains:

```
pointcloud_path     path to .pcd.bin on D:/nuscenes
calibrated_sensor   LiDAR-to-ego transform
ego_pose            ego-to-global transform
```

The raw nuScenes LiDAR point file is a float32 array with 5 values per point:

```
x, y, z, intensity, ring
```

Our current code uses only:

```
x, y, z
```

5.4 `SurroundFrame`

One `SurroundFrame` is the 360 equivalent of a video frame:

```
SurroundFrame(
    scene_token,
    scene_name,
    sample_token,
    sample_index,
    timestamp_us,
    cameras,
    is_key_frame,
)
```

It contains a dictionary of six `CameraFrame` objects:

```
frame.cameras["CAM_FRONT"]
frame.cameras["CAM_BACK_LEFT"]
```

This is the object passed into contact sheet rendering, LiDAR projection, and BEV rendering.

5.5 `NuScenesSceneLoader`

This class owns the dataset root and the metadata tables:

```
loader = NuScenesSceneLoader(dataroot="D:/nuscenes")
```

It defaults to:

```
NUSCENES_ROOT
```

or:

```
D:/nuscenes
```

It does not copy dataset data. It only stores paths to files on D drive.

5.6 Walking through a scene

The scene sequence is a linked list:

```
scene["first_sample_token"]
  -> sample 0
  -> sample["next"]
  -> sample 1
  -> sample["next"]
  -> sample 2
```

The helper `_sample_sequence()` follows this chain and returns samples in order.

If scene 0 has 40 samples:

```
sample 0
sample 1
...
sample 39
```

that becomes a 40-frame keyframe video.

5.7 Why direct JSON reading?

The official `nuscenes-devkit` was not installed in the environment when the loader was created. Instead of blocking, we read the JSON tables directly.

Some metadata files are large, so the loader includes:

```
_iter_json_objects(path)
```

This streams one JSON object at a time from a pretty-printed JSON array. That keeps memory usage lower for large files like `sample_data.json` and `ego_pose.json`.

5.8 Keyframe iteration

The method:

```
iter_scene_frames(..., include_lidar=True)
```

yields:

```
(SurroundFrame, LidarFrame)
```

for each key sample.

This is used by:

```
--view lidar
--view bev
```

5.9 Sweep iteration

The method:

```
iter_camera_sweep_frames(...)
```

creates smoother camera-only frames.

It works like this:

1. Collect all camera records for the scene, including non-keyframes.
2. Use `CAM_FRONT` as the timeline.
3. For each `CAM_FRONT` timestamp, find the nearest image from the other five cameras.
4. Accept the group only if every camera is within the tolerance.

The default tolerance is:

```
100000 microseconds = 100 ms
```

Numerical example:

```
target CAM_FRONT timestamp: 1531883530412470

nearest CAM_FRONT_LEFT:   1531883530404844
nearest CAM_FRONT_RIGHT:  1531883530420339
nearest CAM_BACK:         1531883530437525
nearest CAM_BACK_LEFT:    1531883530447423
nearest CAM_BACK_RIGHT:   1531883530427893
```

The largest difference is around 35 ms, so the group is accepted.

This produces real smooth camera video from sweeps.

6. `fsd/contact_sheet.py`

This file renders six camera images into one visual frame.

The main function is:

```
render_contact_sheet(frame, tile_width=640)
```

It loads each camera image:

```
cv2.imread(str(camera.image_path))
```

Then resizes it to a common width:

```
tile_width
```

and stacks the images:

```
top row:    front-left, front, front-right
bottom row: back-left,  back,  back-right
```

Each tile is labeled with:

```
camera channel
key or sweep
timestamp
```

The header includes:

```
scene name
sample/sweep index
sample token
ego xyz position
```

This is useful because before doing BEV or occupancy, we need confidence that

the six-camera sequence itself is loading correctly.

7. Coordinate Frames

Understanding the coordinate frames is the heart of this pipeline.

nuScenes uses multiple frames:

```
LiDAR sensor frame
camera sensor frame
ego vehicle frame
global/world frame
image pixel frame
BEV grid frame
```

7.1 Ego frame

The ego frame is attached to the vehicle.

Common autonomous-driving convention:

```
x = forward
y = left
z = up
```

So a point:

```
(x=10, y=2, z=0)
```

is 10 meters in front of the car and 2 meters to the left.

7.2 Camera frame

For camera projection, points are represented in the camera coordinate system.

The projection equations assume:

```
Z = depth forward from the camera
X = horizontal coordinate
Y = vertical coordinate
```

A point must have:

```
Z > 0
```

to be in front of the camera.

7.3 Global frame

The global frame is a fixed world coordinate system for the scene. Ego poses

connect each sensor timestamp to this global frame.

This is important for temporal fusion:

```
current ego frame -> global -> next ego frame
```

Later, temporal occupancy mapping will use these poses to warp old grids into

the current vehicle frame.

8. Rigid Body Transforms

nuScenes stores transforms as:

```
translation: [tx, ty, tz]
```

```
rotation: quaternion [w, x, y, z]
```

The quaternion is converted to a 3x3 rotation matrix:

```
R
```

A 3D point is transformed with:

```
p_target = R * p_source + t
```

In row-vector NumPy form, the code uses:

```
points @ R.T + t
```

because points are stored as rows:

```
N x 3
```

8.1 Inverse transform

If:

```
p_target = R * p_source + t
```

then:

```
p_source = R^-1 * (p_target - t)
```

For a rotation matrix:

```
R^-1 = R.T
```

In row-vector form, the code uses:

```
(points - t) @ R
```

8.2 Numerical example

Suppose the sensor is mounted 1 meter forward and 0.5 meters up from ego, with no rotation:

```
R = identity
t = [1.0, 0.0, 0.5]
```

A LiDAR point in sensor coordinates:

```
p_lidar = [10.0, 2.0, 0.0]
```

becomes:

```
p_ego = R * p_lidar + t
      = [10.0, 2.0, 0.0] + [1.0, 0.0, 0.5]
      = [11.0, 2.0, 0.5]
```

Meaning the point is 11 meters forward, 2 meters left, and 0.5 meters up in the ego frame.

9. `fsd/lidar_projection.py`

This file validates calibration by projecting `LIDAR_TOP` points into the six camera images.

The important functions are:

```
load_lidar_points()
quaternion_to_rotation_matrix()
transform_points()
inverse_transform_points()
lidar_points_to_camera()
project_camera_points()
render_lidar_projection_sheet()
```

9.1 Loading LiDAR points

nuScenes LiDAR files are binary float32 arrays:

```
x, y, z, intensity, ring
```

The loader does:

```
points = np.fromfile(path, dtype=np.float32)
points = points.reshape((-1, 5))[:, :3]
```

If the raw file contains 34,720 points, the resulting shape is:

```
34720 x 3
```

9.2 Transform chain: LiDAR to camera

To project LiDAR into a camera image, we need all LiDAR points in the camera sensor frame.

The chain is:

```
LiDAR sensor frame
  -> ego frame at LiDAR timestamp
  -> global frame
  -> ego frame at camera timestamp
  -> camera sensor frame
```

In equations:

```
p_ego_lidar = R_lidar_to_ego * p_lidar + t_lidar_to_ego
p_global = R_ego_lidar_to_global * p_ego_lidar + t_ego_lidar_to_global
p_ego_camera = inverse(T_ego_camera_to_global) * p_global
p_camera = inverse(T_camera_to_ego) * p_ego_camera
```

The code:

```
points = transform_points(
    lidar_points,
    lidar.calibrated_sensor["rotation"],
    lidar.calibrated_sensor["translation"],
)

points = transform_points(
    points,
    lidar.ego_pose["rotation"],
    lidar.ego_pose["translation"],
)

points = inverse_transform_points(
    points,
    camera.ego_pose["rotation"],
    camera.ego_pose["translation"],
)

points = inverse_transform_points(
    points,
    camera.calibrated_sensor["rotation"],
    camera.calibrated_sensor["translation"],
)
```

This is the most important math in the current system.

9.3 Camera projection

Once a point is in camera coordinates:

```
p_camera = [X, Y, Z]
```

it is projected using:

```
u = fx * X / Z + cx
v = fy * Y / Z + cy
```

where:

```
u, v = image pixel coordinates
fx, fy = focal lengths
cx, cy = principal point
Z = depth in camera frame
```

Matrix form:

```
[u']   [fx  0  cx] [X]
[v'] = [0  fy  cy] [Y]
[w']   [0   0   1] [Z]
```

```
u = u' / w'
v = v' / w'
```

9.4 Numerical projection example

Suppose:

```
fx = 1000
fy = 1000
cx = 800
cy = 450
```

and a camera-frame point is:

```
X = 2 m
Y = 1 m
Z = 20 m
```

Then:

```
u = 1000 * 2 / 20 + 800 = 900
v = 1000 * 1 / 20 + 450 = 500
```

So the LiDAR point lands at pixel:

```
(900, 500)
```

If the image is 1600x900, this point is visible. If `u` or `v` is outside the image bounds, the point is discarded.

9.5 Depth filtering

The code requires:

```
depth > min_depth
```

with default:

```
min_depth = 1.0 meter
```

This removes points behind the camera or too close to be useful.

9.6 Coloring points

Projected points are colored by depth:

```
cv2.COLORMAP_TURBO
```

Near and far points get different colors. This makes it easier to visually inspect whether the projected LiDAR follows the road, vehicles, and buildings in the camera image.

9.7 Calibration smoke test

For scene 0, sample 0:

```
raw LiDAR points: 34720
CAM_FRONT_LEFT: 3263 projected points
CAM_FRONT: 3364 projected points
CAM_FRONT_RIGHT: 2770 projected points
CAM_BACK_LEFT: 4055 projected points
CAM_BACK: 4318 projected points
CAM_BACK_RIGHT: 2998 projected points
```

This means thousands of points projected into each camera image, which is a good first sign that the transform chain is working.

The output file was:

```
outputs/nuscenes_lidar_projection_scene0_sample0.jpg
```

10. `fsd/bev.py`

This file renders an ego-frame bird's-eye view from `LIDAR_TOP`.

Important functions:

```
lidar_points_to_ego()
```

```
render_lidar_bev()
```

```
save_lidar_bev_sequence()
```

10.1 What BEV means here

BEV means bird's-eye view: a top-down representation around the ego vehicle.

The source is 3D LiDAR:

```
x, y, z
```

The output is currently a 2D top-down image:

```
row, col
```

So this is:

```
3D points -> 2D BEV visualization
```

It is not yet full 3D voxel occupancy.

10.2 LiDAR to ego

For BEV, we do not need to go through global or camera frames. We only need:

```
LiDAR sensor frame -> ego frame
```

The code:

```
def lidar_points_to_ego(lidar_points, lidar):
    return transform_points(
        lidar_points,
        lidar.calibrated_sensor["rotation"],
        lidar.calibrated_sensor["translation"],
    )
```

After this, each point is in ego coordinates:

```
x forward
```

```
y left
```

```
z up
```

10.3 BEV ranges

The default BEV covers:

```
x_range = [-50, 50] meters
y_range = [-50, 50] meters
z_range = [-3, 5] meters
resolution = 0.25 meters per cell
```

This means:

```
100 m forward/backward
100 m left/right
```

At 0.25 m per cell:

```
100 / 0.25 = 400 cells
```

So the raw BEV grid is:

```
400 x 400
```

The renderer adds a header and then scales the output by 2, so the saved video

is:

```
800 x 916
```

10.4 Metric to pixel mapping

The code maps ego-frame metric coordinates to image pixels:

```
cols = ((y_max - y) / resolution)
rows = ((x_max - x) / resolution)
```

This convention means:

```
forward x increases upward in the image
left y increases leftward in the image
ego vehicle is in the center
```

For:

```
x_range = [-50, 50]
y_range = [-50, 50]
resolution = 0.25
```

the ego origin:

```
x = 0, y = 0
```

maps to:

```
row = (50 - 0) / 0.25 = 200
col = (50 - 0) / 0.25 = 200
```

So ego is at pixel:

```
(row=200, col=200)
```

before scaling.

10.5 Numerical BEV example

Suppose a point is:

```
x = 12.3 m forward

y = -4.7 m right

resolution = 0.25 m/cell

x_max = 50

y_max = 50
```

Then:

```
row = (50 - 12.3) / 0.25

    = 37.7 / 0.25

    = 150.8

    -> 150


col = (50 - (-4.7)) / 0.25

    = 54.7 / 0.25

    = 218.8

    -> 218
```

The point appears above and slightly to the right of ego, which matches:

```
forward and right
```

10.6 Height filtering

The renderer uses:

```
z_range = [-3, 5]
```

Points outside this vertical range are ignored.

This removes extremely low or high points and keeps the BEV focused on road,

vehicles, poles, buildings, and other relevant structures.

10.7 Height coloring

The code colors points by height:

```
z_norm = (z - z_min) / (z_max - z_min)

cv2.applyColorMap(..., cv2.COLORMAP_TURBO)
```

So low and high points get different colors. This is not occupancy yet. It is

a visual diagnostic.

10.8 Ego marker

The BEV draws a small ego vehicle at the center:

```
rectangle = car footprint

arrow     = forward direction
```

This makes it clear which way the car is facing.

10.9 BEV output

The unified BEV output is:

```
outputs/nuscenes_scene0_40f_bev_unified.mp4

40 frames

2 FPS

20.0 seconds

800x916
```

11. `fsd/visualize.py`

This is now the canonical visualization entry point.

Use:

```
python -m fsd.visualize
```

The supported views are:

```
--view cameras

--view lidar

--view bev
```

11.1 Camera view

```
.\.venv\Scripts\python.exe -m fsd.visualize --dataroot D:/nuscenes --scene-index 0 --frames 5 --view

cameras --save --tile-width 360 --fps 2 --output outputs/nuscenes_cameras_sequence_smoke.mp4
```

This renders six camera images per frame.

11.2 Smooth camera sweep view

```
.\.venv\Scripts\python.exe -m fsd.visualize --dataroot D:/nuscenes --scene-index 0 --frames 233 --view

 cameras --sequence sweeps --save --tile-width 360 --fps 12 --output

outputs/nuscenes_scene0_camera_sweeps_12fps.mp4
```

This uses camera sweeps instead of sparse key samples.

Output:

```
233 frames
12 FPS
19.4 seconds
1080x480
```

11.3 LiDAR projection view

```
.\.venv\Scripts\python.exe -m fsd.visualize --dataroot D:/nuscenes --scene-index 0 --frames 40 --view
lidar --save --tile-width 480 --fps 2 --output outputs/nuscenes_scene0_40f_lidar.mp4
```

This projects LiDAR points into camera images and creates a six-camera overlay

video.

11.4 BEV view

```
.\.venv\Scripts\python.exe -m fsd.visualize --dataroot D:/nuscenes --scene-index 0 --frames 40 --view
bev --sequence keyframes --save --fps 2 --bev-resolution 0.25 --bev-scale 2 --output
outputs/nuscenes_scene0_40f_bev_unified.mp4
```

This renders the ego-frame LiDAR BEV.

11.5 Save vs stream

For video output:

```
--save
```

For live OpenCV display:

```
--stream
```

In stream mode, press:

```
q
```

to quit.

11.6 Sequence modes

The visualizer supports:

```
--sequence keyframes
--sequence sweeps
```

Current support:

```
cameras + keyframes: yes
cameras + sweeps:    yes
lidar + keyframes:   yes
bev + keyframes:     yes
lidar + sweeps:      not yet
```

```
bev + sweeps:        not yet
```

LiDAR sweeps need a synchronization policy before we use them heavily:

```
nearest LiDAR sweep
```

```
previous LiDAR sweep
```

```
accumulated LiDAR sweeps
```

```
interpolated ego poses
```

For now, keeping LiDAR and BEV on keyframes is the cleaner validation path.

12. How Projection and BEV Relate

The LiDAR projection view and the BEV view use the same underlying geometry,

but they answer different questions.

12.1 Projection view

Question:

```
Do the LiDAR points line up with camera images?
```

Transform chain:

```
LiDAR -> ego -> global -> camera ego -> camera -> image pixels
```

Output:

```
camera images with LiDAR dots
```

This validates calibration.

12.2 BEV view

Question:

```
What does the world look like from above in ego coordinates?
```

Transform chain:

```
LiDAR -> ego -> BEV grid pixels
```

Output:

```
top-down LiDAR map
```

This validates the top-down coordinate convention.

12.3 Occupancy grid, next

Question:

```
Which cells are free, occupied, or unknown?
```

Transform chain:

```
LiDAR -> ego -> raycast occupancy grid
```

Output:

```
grid[x, y] = free / occupied / unknown
```

This becomes the world model.

13. What Is Not Implemented Yet

Current state:

```
done: dataset loader

done: six-camera contact sheet

done: smooth camera sweep visualization

done: LiDAR-to-camera projection

done: ego-frame LiDAR BEV visualization
```

Not yet:

```
2D occupancy grid

raycasting free space

log-odds fusion

temporal BEV warping

camera depth back-projection

drivable-space BEV from segmentation

object velocity estimation

lane topology

neural BEV models
```

14. Next Step: Occupancy Grid

The next major file should be:

```
fsd/occupancy.py
```

It should convert one LiDAR scan into a real occupancy grid:

```
unknown

free

occupied
```

The BEV image we have now is visualization. Occupancy is data.

14.1 Occupancy grid representation

Use a 2D array:

```
log_odds = np.zeros((height, width), dtype=np.float32)
```

```
observed = np.zeros((height, width), dtype=bool)
```

Each cell stores a belief:

```
positive log-odds = occupied
```

```
negative log-odds = free
```

```
near zero         = unknown
```

14.2 Log-odds

Probability:

```
p = probability cell is occupied
```

Log-odds:

```
L = log(p / (1 - p))
```

If:

```
p = 0.5
```

then:

```
L = log(0.5 / 0.5) = log(1) = 0
```

So zero means unknown.

If:

```
p = 0.7
```

then:

```
L = log(0.7 / 0.3) = log(2.333) = 0.847
```

Positive means likely occupied.

If:

```
p = 0.3
```

then:

```
L = log(0.3 / 0.7) = log(0.428) = -0.847
```

Negative means likely free.

14.3 Raycasting

For every LiDAR point:

```
ego origin -> endpoint
```

Cells along the ray are free. The endpoint cell is occupied.

Example:

```
ego at (0, 0)
LiDAR point at (10, 2)
```

The ray says:

```
cells from (0, 0) to near (10, 2): free
cell at (10, 2): occupied
```

This is much better than just plotting points, because it gives us explicit free space.

14.4 Temporal fusion

Once we have occupancy per frame, we can fuse over time:

```
grid at t-1
  -> warp using ego motion
  -> align with current ego frame
  -> add current occupancy evidence
  -> decay old uncertain cells
```

This is the "persistent world model" described in `plan.md`.

15. Why 2D Before 3D

The LiDAR source is 3D:

```
x, y, z
```

But the next occupancy grid should be 2D:

```
grid[x, y]
```

That is enough for the first planning-oriented world model:

```
Can the car drive here?
```

```
Is there an obstacle here?
What nearby space is unknown?
```

Full 3D occupancy is:

```
voxel[x, y, z]
```

It is more expensive.

Numerical comparison at 0.25 m resolution:

```
2D grid:
100m x 100m = 400 x 400 = 160,000 cells

3D grid:
100m x 100m x 8m = 400 x 400 x 32 = 5,120,000 voxels
```

So 3D is about 32 times larger for this volume.

The practical path is:

```
2D occupancy
  -> 2.5D height-aware occupancy
  -> 3D voxel occupancy
```

16. Current File Responsibilities

Summary by file:

```
fsd/data.py
```

Dataset access, metadata parsing, scene iteration, keyframe iteration, camera sweep grouping, `CameraFrame`, `LidarFrame`, `SurroundFrame`.

```
fsd/contact_sheet.py
```

Six-camera image layout rendering.

```
fsd/lidar_projection.py
```

LiDAR loading, quaternion math, rigid transforms, LiDAR-to-camera projection, camera overlay rendering.

```
fsd/bev.py
```

LiDAR-to-ego transform and ego-frame top-down BEV rendering.

```
fsd/visualize.py
```

Unified CLI for:

```
cameras
lidar
bev
```

and:

```
keyframes
sweeps
save
stream
```

17. Key Commands

Smooth camera sweep video:

```
.\.venv\Scripts\python.exe -m fsd.visualize --dataroot D:/nuscenes --scene-index 0 --frames 233 --view
 cameras --sequence sweeps --save --tile-width 360 --fps 12 --output
outputs/nuscenes_scene0_camera_sweeps_12fps.mp4
```

LiDAR projection video:

```
.\.venv\Scripts\python.exe -m fsd.visualize --dataroot D:/nuscenes --scene-index 0 --frames 40 --view
lidar --sequence keyframes --save --tile-width 480 --fps 2 --output
outputs/nuscenes_scene0_40f_lidar.mp4
```

BEV video:

```
.\.venv\Scripts\python.exe -m fsd.visualize --dataroot D:/nuscenes --scene-index 0 --frames 40 --view
bev --sequence keyframes --save --fps 2 --bev-resolution 0.25 --bev-scale 2 --output
outputs/nuscenes_scene0_40f_bev_unified.mp4
```

Live BEV stream:

```
.\.venv\Scripts\python.exe -m fsd.visualize --dataroot D:/nuscenes --scene-index 0 --frames 40 --view
bev --stream
```

18. Final Mental Model

The current 360 pipeline is not "a neural autonomous driver" yet. It is the foundation for one.

What we have now:

```
nuScenes metadata
  -> synchronized six-camera frames
  -> camera sweeps for smoother video
```

```
 -> LiDAR point cloud loading
 -> LiDAR-to-camera projection
 -> ego-frame LiDAR BEV
 -> unified visualization
```

What this gives us:

```
correct sequencing
correct calibration use
correct coordinate transforms
correct top-down convention
visual debugging artifacts
```

What comes next:

```
LiDAR BEV visualization
 -> LiDAR occupancy grid
 -> free/occupied/unknown cells
 -> temporal fusion using ego poses
 -> persistent world model
```

That is the core of Phase 1 world modeling.